
TECHNOLOGY & AI DELIVERY

TECHNICAL ASSESSMENT

Document Conversion & Splitting Service

ROLE: SENIOR FULL STACK ENGINEER

Role	Senior Full Stack Engineer — Angular / .NET
Assignment	Document Conversion & Splitting Service
Submission Deadline	4 calendar days from the date this document is sent to you
Required Stack	Frontend: Angular Backend: .NET
AI Tools	Allowed for writing code — you must be able to explain and modify the logic live
Libraries	Any free or trial library/package is allowed
Demo	You will run your own code live during the interview

What "Senior" Means for This Assessment

- We're evaluating architectural judgment, not just working code — how you structure boundaries between API, domain logic, and persistence.
- Trade-off reasoning matters as much as the solution: be ready to explain why you chose an approach over the alternatives.
- Ownership of AI-assisted code — you should be able to defend, modify, and extend any part live, regardless of how it was written.
- Depth over breadth: a smaller set of edge cases handled rigorously outweighs a longer feature list handled loosely.

1. Purpose of This Assessment

This is a take-home technical case study, not a puzzle with one "correct" answer. It is designed to see how you think: how you break down a problem, structure a solution, handle exceptions and edge cases, and how far you push the design when you have room to do more. Most of the technical interview will be built around the code you submit, so treat this as the start of a conversation rather than a final exam.

You are welcome to use AI coding assistants while building this. What matters is that you understand and can defend every part of the logic — you will be asked to explain your design choices and to change or extend the code live during the interview.

2. Technology Stack

- Frontend: Angular (any recent version).
- Backend: .NET (any recent version — Web API is expected).
- Any free or trial library or package is allowed for either side of the stack.

3. Business Scenario

A back-office system needs to prepare documents for handoff to an external recipient. Documents are retrieved from a source in one file format, but the recipient sometimes requires the file in a different format. Once converted, if the resulting file is larger than an agreed size limit, it must be split into multiple sequential parts before it is handed off. Every request needs to be trackable end to end — what was requested, what happened to it, and whether it completed successfully or was rejected.

This mirrors a real assignment we work on, generalized here so the exercise stands on its own without any client-specific detail.

4. Functional Requirements

4.1 Document Intake

- Accept a source document (assume PDF as the input format) along with the requested output format.
- Provide a handful of sample PDFs of your own for testing — a mix of text-based and image-only/scanned pages is a good idea.

4.2 Conversion

- Convert the document from the source format into the requested output format (choose a reasonable target, e.g. PDF to DOCX or PDF to a structured text/HTML format — state your choice in the README).
- Conversion should be rule-based/deterministic, not OCR or AI-based extraction: text should be carried across faithfully, and any image, logo, or signature should be copied into the output as-is rather than interpreted.
- A document that is only a scanned image (no extractable text layer) cannot be converted by these rules — it should be raised as an exception, not silently skipped.

4.3 Size-Based Splitting

- If the converted output exceeds a configurable size limit (default: 2 MB), split it into multiple ordered parts.
- Each part should be clearly sequenced (e.g. part 1 of 3, part 2 of 3...) so the full set can be reassembled or accounted for downstream.
- Think through the edge cases: output exactly at the limit, a single unsplitable element larger than the limit, an empty document, etc.

4.4 Validation

- Before a job is marked complete, verify the output is complete and consistent — e.g. the right number of parts were produced, and nothing was lost or duplicated in the process.
- A job that fails validation should not be marked as successful — it should be flagged for review instead.

4.5 Exception Handling

- Handle, at minimum: unsupported/corrupted input files, unsupported requested formats, and scanned/image-only documents.
- Exceptions should be visible in the job's status/history, not just logged to a console and forgotten.

4.6 Job Tracking & History

- Persist a record for every request: status, timestamps, and outcome (including exceptions).
- The Angular frontend should let a user submit a new request and view the status/history of past ones.

5. Non-Functional Expectations

- Reasonable separation of concerns (e.g. API layer, conversion/splitting logic, and persistence should not all live in one file).
- Deliberate error handling — not just try/catch blocks that swallow everything.
- At least a handful of automated tests around the conversion, splitting, and validation logic — this is one of the areas we look at most closely.
- Code that reads clearly matters more than the number of features covered.

6. What We're Evaluating

Area	What We Are Evaluating
Logic & Correctness	Does the core conversion/splitting/validation logic actually do what it claims, including at the edges?
Scenario Coverage	How many realistic scenarios and exceptions did you think to handle, beyond the happy path?
Complexity Reached	How far did you take the design in the time available, including architectural decisions expected of a senior engineer — what did you prioritize and why?
Problem-Solving	Your ability to explain trade-offs, reasoning, and alternatives you considered.
Code Quality	Structure, readability, and testing — not visual polish of the UI.
Ownership	Your ability to explain, defend, and extend your own code live, including parts written with AI assistance.

7. Constraints

- Angular and .NET are mandatory for frontend and backend respectively.
- Free or trial libraries and packages are allowed on either side.
- AI coding assistants are allowed — there is no restriction on how you use them to write code, but you must understand the result.
- No integration with any real external system is required — simulate inputs and outputs locally (e.g. sample files on disk, an in-memory or local database).

8. Deliverables & Submission

- Source code for both the Angular frontend and the .NET backend (a Git repository link or a zip archive is fine).
- A README with clear setup and run instructions.

- A short note (a few lines is enough) on any assumptions you made and anything you'd do differently with more time.

Submission Deadline

- Please submit within 4 calendar days of receiving this document.

9. Interview Format

- You will run your own code live during the interview — please come prepared with your environment ready to go.
- Most of the technical interview will be built around this task: expect questions on your design decisions, requests to walk through specific parts of the logic, and possibly a small live extension to an existing scenario.

Good luck — we're looking forward to seeing how you approach it.